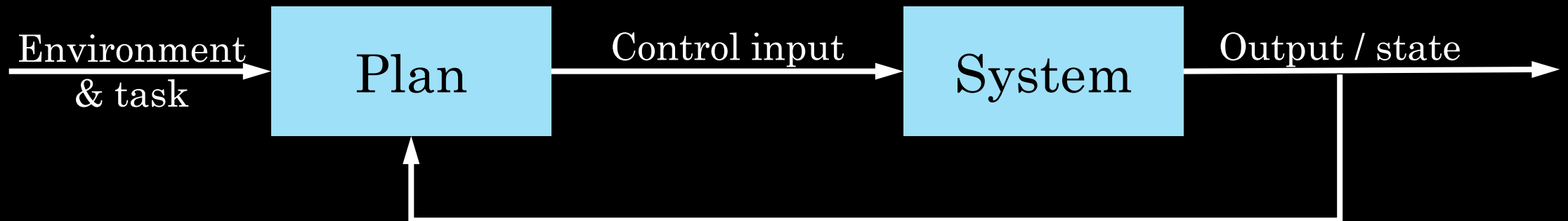


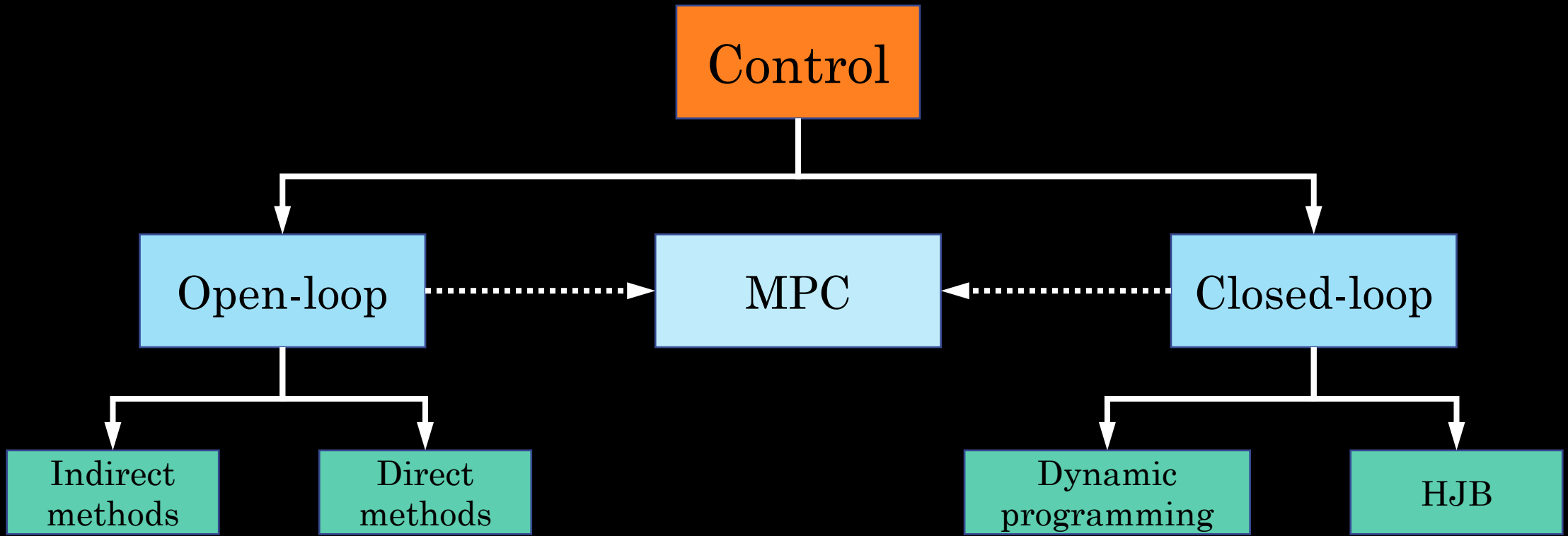
Closed-loop vs open-loop control

Open-loop control: No feedback loop



Closed-loop control: Yes feedback loop





Sequential decision making

Dynamic programming // Closed-loop control

Additional reference: <https://underactuated.mit.edu/dp.html>

How should we make optimal decisions which have consequences on the future?

State = [academic, professionalism, well-being, finances]



Study every day

Study every day

Study every day

Study every day

Apply for job

Join research lab

Join research lab

Join research lab

Join research lab

Apply to grad school

Attend parties

Attend parties

Attend parties

Attend parties

Travel the world

Work two jobs

Work two jobs

Work two jobs

Work two jobs

Create start-up

Overcommit to RSO

Overcommit to RSO

Overcommit to RSO

Overcommit to RSO

Attend parties

**First day of
school**

**End of
Freshman**

**End of
Sophomore**

**End of
Junior**

**End of
Senior**

How should we make optimal decisions which have consequences on the future?

State = [academic, professionalism, well-being, finances]

Study every day

Apply for job

Join research lab

Apply to grad school

Attend parties

Travel the world

Work two jobs

Create start-up

Overcommit to RSO

Attend parties

**End of
Senior**

Retirement

Sequential decision making

- **Actions have consequences.** The consequences may be immediate or delayed.
 - E.g., Attending party before exam
 - Immediate: Lots of fun, improve mood and happiness
 - Delayed: Not alert during exam, perform poorly, not get a good grade, miss out on Dean's list.
- Hard to make optimal decisions
 - Uncertainty in future (e.g., exam may be easy)
 - Large state space (many possible situations could arise)
 - Hard to measure exactly the consequences

Group activity

- Describe some “real-world” settings where you need to perform sequential decision making
 - Immediate consequence
 - Future consequence
 - States and decisions/controls
 - Potential sources of uncertainties, and potential ways to address them
- Example: Chess
 - Taking opponent’s pieces
 - Setting up board to be in a position to win
 - Location of pieces
 - Piece to move
 - What the opponent does on their move
 - Could assume opponent is optimal

Are there SDM problems with nice structure that make it easier to solve?

Principle of Optimality

- An optimal solution contains within it optimal solutions to its subproblems
- Given an optimal solution to a SDM problem, then any tail portion of the solution is also optimal to that sub SDM problem
- When following an optimal solution, the remainder of the solution must also be optimal from that state onwards.
- If the problem is solved optimally, then regardless of past decisions, future decisions must be optimal for the current state.

How should we make optimal decisions which have consequences on the future?

State = [academic, professionalism, well-being, finances]



Study every day

Study every day

Study every day

Study every day

Apply for job

Join research lab

Join research lab

Join research lab

Join research lab

Apply to grad school

Attend parties

Attend parties

Attend parties

Attend parties

Travel the world

Work two jobs

Work two jobs

Work two jobs

Work two jobs

Create start-up

Overcommit to RSO

Overcommit to RSO

Overcommit to RSO

Overcommit to RSO

Attend parties

**First day of
school**

**End of
Freshman**

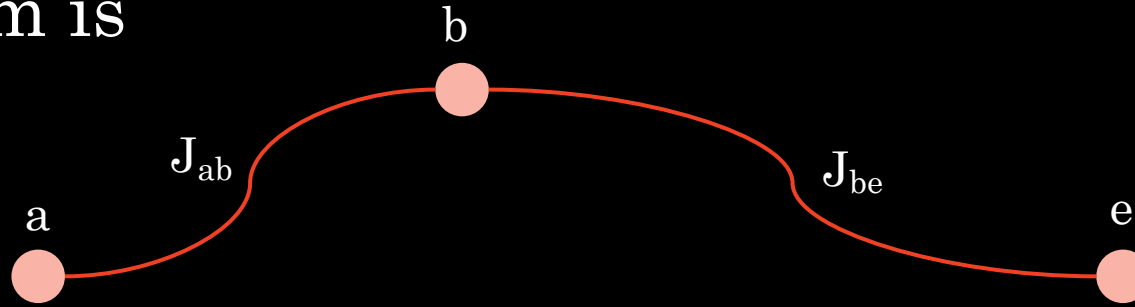
**End of
Sophomore**

**End of
Junior**

**End of
Senior**

Principle of optimality: Setup

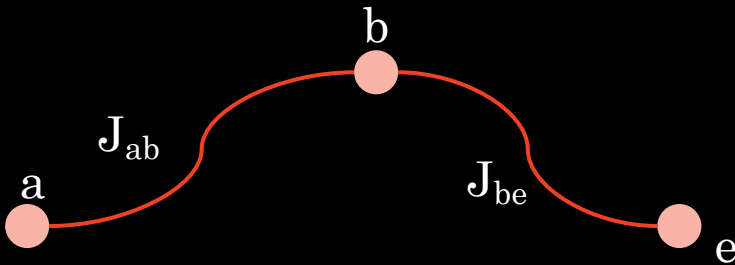
- Suppose optimal path for a multi-stage decision-making problem is



- First decision yields segment $a - b$ with cost J_{ab}
- Remaining decisions yield segment $b - e$ with cost J_{be}
- Optimal cost is then $J_{ae}^* = J_{ab} + J_{be}$
- **Assumption:** Cost has *additive* structure

Principle of optimality

- **Claim:** If $a - b - e$ is the optimal path from a to e , then $b - e$ is an optimal path from b to e
- **Proof:** By contradiction



Implications of the Principle of Optimality

- To find optimal decision sequence, we consider solving tail subproblems.
- Then consider solving the tail subproblems of those subproblems, and so forth.
- Start with the smallest tail subproblem and work our way backwards
- Can ignore the past, just concerned with optimizing the future from the current situation
 - Need to solve for *all* tail subproblems

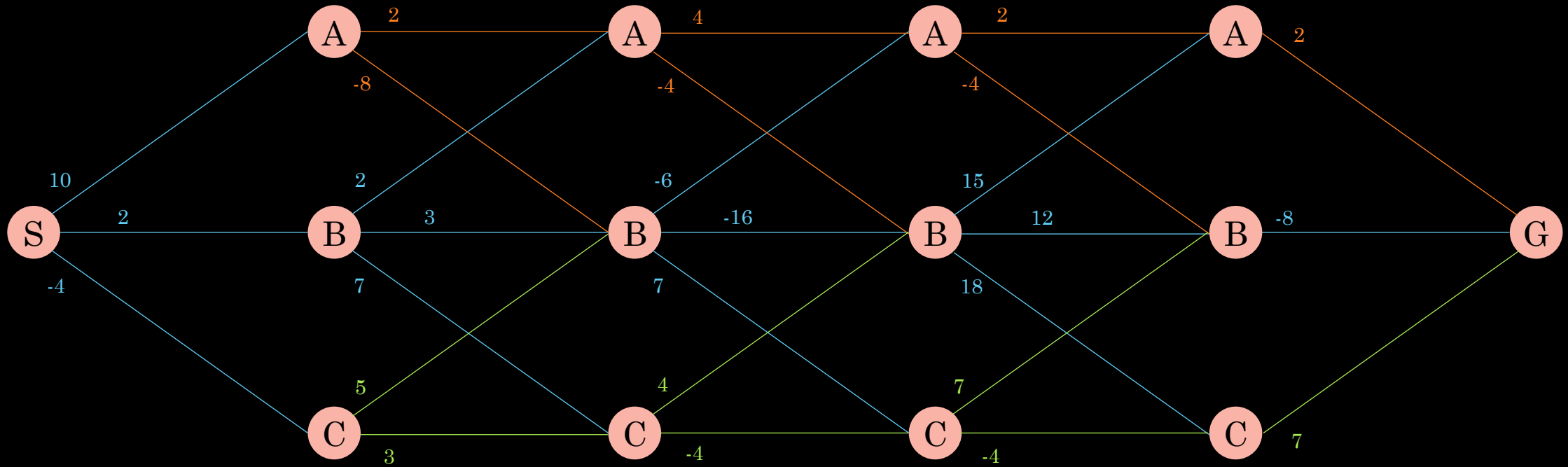
Dynamic programming

- Algorithmic technique for solving complex problems by breaking them down into simpler subproblems
- **Core idea:** Solve each subproblem only once and store the solution
- **Benefit:** Prevents redundant computations when subproblem is encountered again
- A common technique to solve many problems, not just control problems

Example of dynamic programming in other fields

-  Dividing text of paragraph into lines of approximately equal length—Knuth–Plass line-breaking algorithm used dynamic programming
-  Route optimization, traveling salesman problem, orienteering problem. Useful for delivery route planning, surveillance missions, transportation routing, etc
-  Sequence alignment: Comparing DNA or protein sequences to find similarities (e.g., Needleman–Wunsch or Smith–Waterman algorithms).

Example



Key idea: Keep track of a *value function*

Value function

A “score” that measures the long-term potential of that state, assuming a fixed policy π is used to go from that point forward.

- “Cost-to-go”: The future accumulated cost (or reward) from following policy π
- Combines immediate and future consequences

Trajectory optimization as a SDM

Consider the discrete time setting (easier to start with)

- **System:** $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k, t_k), k = 0, \dots, N - 1$
- **Control constraints:** $\mathbf{u}_k \in U(\mathbf{x}_k)$.
- **Cost:** $J(\mathbf{x}_0, \mathbf{u}_{0:N-1}, t_0) = g_N(\mathbf{x}_N) + \sum_{k=0}^{N-1} g(\mathbf{x}_k, \mathbf{u}_k, t_k)$
- Let $\{\mathbf{u}_0^*, \mathbf{u}_1^*, \dots, \mathbf{u}_{N-1}^*\}$ be an *optimal* control sequence starting at state $\mathbf{x}_0$, with corresponding state sequence $\{\mathbf{x}_0^*, \mathbf{x}_1^*, \dots, \mathbf{x}_N^*\}$

Take a closer look at the cost

Let $\mathbf{u}_k^* = \pi^*(\mathbf{x}_k, t_k)$ be the optimal policy

Cost at t_0 ($k = 0$)

$$J^*(\mathbf{x}_0, t_0) = g_N(\mathbf{x}_N) + \sum_{k=0}^{N-1} g(\mathbf{x}_k, \pi^*(\mathbf{x}_k, t_k), t_k)$$

By Principle of Optimality

$$J^*(\mathbf{x}_j, t_j) = g_N(\mathbf{x}_N) + \sum_{k=j}^{N-1} g(\mathbf{x}_k, \pi^*(\mathbf{x}_k, t_k), t_k)$$

$$J^*(\mathbf{x}_j, t_j) = g(\mathbf{x}_j, \pi^*(\mathbf{x}_j, t_j), t_j) + g_N(\mathbf{x}_N) + \sum_{k=j+1}^{N-1} g(\mathbf{x}_k, \pi^*(\mathbf{x}_k, t_k), t_k)$$

$$J^*(\mathbf{x}_j, t_j) = g(\mathbf{x}_j, \pi^*(\mathbf{x}_j, t_j), t_j) + J^*(\mathbf{x}_{j+1}, t_{j+1})$$

Value function represents *cost-to-go*

Cost-to-go, also referred to as the **Value Function**

- The future total cost at $(\mathbf{x}_k, t_k)$ if following policy π

$$V_{\pi}(\mathbf{x}_k, t_k) = g_N(\mathbf{x}_N) + \sum_{k=k}^{N-1} g(\mathbf{x}_k, \pi(\mathbf{x}_k, t_k), k) := J(\mathbf{x}_j, t_j; \pi)$$

$$J(\mathbf{x}_j, t_j; \pi) = g(\mathbf{x}_j, \pi(\mathbf{x}_j, t_j), t_j) + J(\mathbf{x}_{j+1}, t_{j+1}; \pi)$$

$$V_{\pi}(\mathbf{x}_k, t_k) = g(\mathbf{x}_k, \pi(\mathbf{x}_k, t_k), t_k) + V_{\pi}(\mathbf{x}_{k+1}, t_{k+1})$$

How to compute the *optimal* policy?

Previously, it was assumed a policy was provided.

We can simultaneously compute the optimal policy while computing the value function.

$$V_{\pi}(\mathbf{x}_k, t_k) = g(\mathbf{x}_k, \pi(\mathbf{x}_k, t_k), t_k) + V_{\pi}(\mathbf{x}_{k+1}, t_{k+1})$$

$$V^*(\mathbf{x}_k, t_k) = \min_{\mathbf{u}_k \in \mathcal{U}} g(\mathbf{x}_k, \mathbf{u}_k, t_k) + V^*(f(\mathbf{x}_k, \mathbf{u}_k, t_k), t_{k+1})$$

Bellman equation

$$V^*(\mathbf{x}_k, t_k) = \min_{\mathbf{u}_k \in \mathcal{U}} g(\mathbf{x}_k, \mathbf{u}_k, t_k) + V^*(f(\mathbf{x}_k, \mathbf{u}_k, t_k), t_{k+1})$$

Immediate cost

Future cost

Okay, but how exactly do we compute the Value Function?

Assumes we have V at the next time step

$$V_{\pi}(\mathbf{x}_k, t_k) = g(\mathbf{x}_k, \pi(\mathbf{x}_k, t_k), t_k) + V_{\pi}(\mathbf{x}_{k+1}, t_{k+1})$$

$$V^*(\mathbf{x}_k, t_k) = \min_{\mathbf{u}_k \in \mathcal{U}} g(\mathbf{x}_k, \mathbf{u}_k, t_k) + V^*(f(\mathbf{x}_k, \mathbf{u}_k, t_k), t_{k+1})$$

Need a *terminal* case (aka base case)

When $k = N$?

$$V^*(\mathbf{x}_N, t_N) = g_N(\mathbf{x}_N)$$

Base case: Use terminal cost

$$V^*(\mathbf{x}_N, t_N) = g_N(\mathbf{x}_N)$$

$$V^*(\mathbf{x}_{N-1}, t_{N-1}) = \min_{\mathbf{u}_{N-1} \in \mathcal{U}} g(\mathbf{x}_{N-1}, \mathbf{u}_{N-1}, t_{N-1}) + V^*(f(\mathbf{x}_{N-1}, \mathbf{u}_{N-1}, t_{N-1}), t_N)$$

$$V^*(\mathbf{x}_{N-2}, t_{N-2}) = \min_{\mathbf{u}_{N-2} \in \mathcal{U}} g(\mathbf{x}_{N-2}, \mathbf{u}_{N-2}, t_{N-2}) + V^*(f(\mathbf{x}_{N-2}, \mathbf{u}_{N-2}, t_{N-2}), t_{N-1})$$

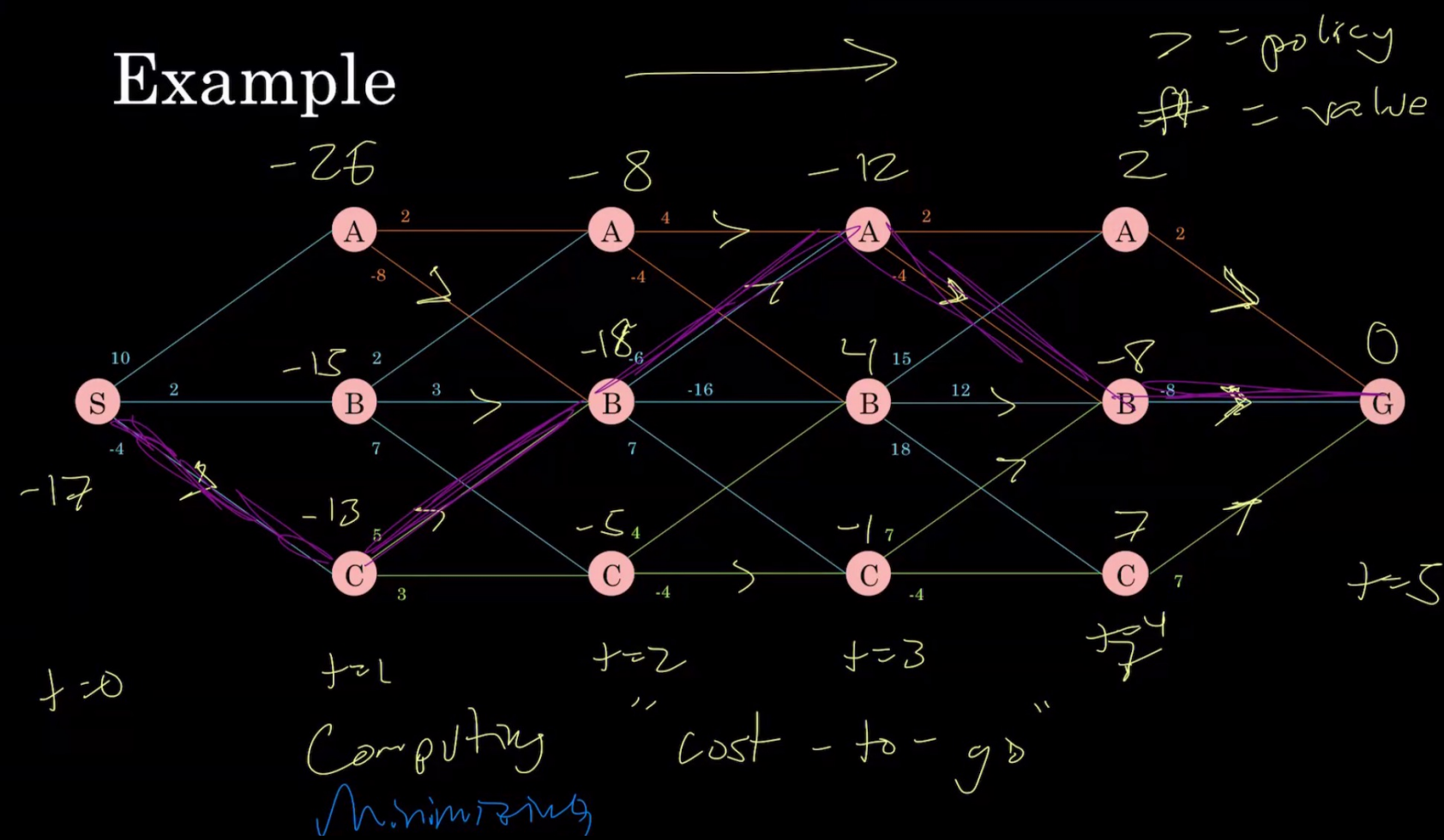
...

$$V^*(\mathbf{x}_0, t_0) = \min_{\mathbf{u}_0 \in \mathcal{U}} g(\mathbf{x}_0, \mathbf{u}_0, t_0) + V^*(f(\mathbf{x}_0, \mathbf{u}_0, t_0), t_0)$$

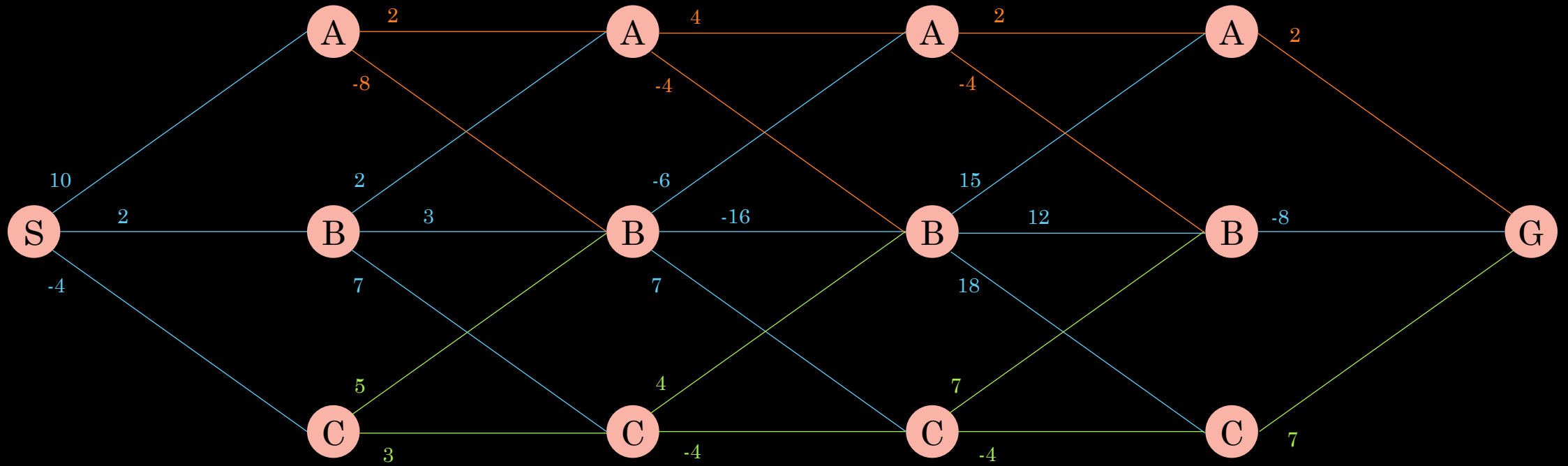
Dynamic programming: Summary

- Work *backward* in time
- Start with terminal cost: $V^*(\mathbf{x}_N, t_N) = g_N(\mathbf{x}_N)$
- Sweep backward in time
 - $V^*(\mathbf{x}_k, t_k) = \min_{\mathbf{u}_k \in \mathcal{U}} g(\mathbf{x}_k, \mathbf{u}_k, t_k) + V^*(f(\mathbf{x}_k, \mathbf{u}_k, t_k), t_{k+1})$
 - $\pi^*(\mathbf{x}_k, t_k) = \operatorname{argmin}_{\mathbf{u}_k \in \mathcal{U}} g(\mathbf{x}_k, \mathbf{u}_k, t_k) + V^*(f(\mathbf{x}_k, \mathbf{u}_k, t_k), t_{k+1})$
- Final outputs
 - $V^*(\mathbf{x}_k, t_k)$ for all states and all time
 - $\pi^*(\mathbf{x}_k, t_k)$ for all states and all time

Example



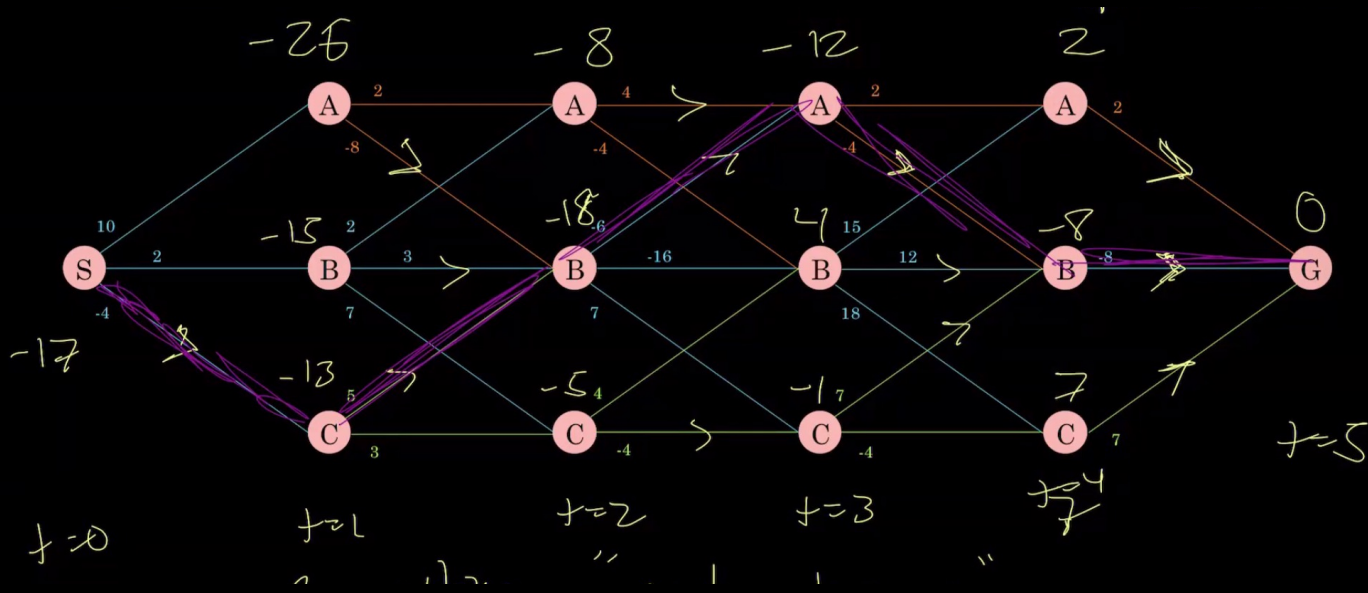
$$V^*(\mathbf{x}_k, t_k) = \min_{\mathbf{u}_k \in \mathcal{U}} g(\mathbf{x}_k, \mathbf{u}_k, t_k) + V^*(f(\mathbf{x}_k, \mathbf{u}_k, t_k), t_{k+1})$$



$$V^*(\mathbf{x}_k, t_k) = \min_{\mathbf{u}_k \in \mathcal{U}} g(\mathbf{x}_k, \mathbf{u}_k, t_k) + V^*(f(\mathbf{x}_k, \mathbf{u}_k, t_k), t_{k+1})$$

Storing the value function

Discrete time, state & control



π	t_0	t_1	t_2	t_3	t_4
S					
A					
B					
C					

V	t_0	t_1	t_2	t_3	t_4
S					
A					
B					
C					

Direct trajectory optimization vs dynamic programming

Both aim to find $\{\mathbf{u}_0^*, \mathbf{u}_1^*, \dots, \mathbf{u}_{N-1}^*\}$ that minimizes total cost $g_N(\mathbf{x}_N) + \sum_{k=0}^{N-1} g(\mathbf{x}_k, \mathbf{u}_k, t_k)$, subject to dynamics and constraints.

Direct trajectory optimization:

Dynamic programming:

The Bellman Equation and variants

Stochastic setting

- Stochastic dynamics: $\mathbf{x}_{k+1} \sim f(\mathbf{x}_k, \mathbf{u}_k, t_k)$
- Cost may depend on next state too: $g(\mathbf{x}_k, \mathbf{u}_k, \mathbf{x}_{k+1}, t_k)$

$$V^*(\mathbf{x}_k, t_k) = \min_{\mathbf{u}_k \in \mathcal{U}} \mathbb{E}_{\mathbf{x}_{k+1} \sim f(\mathbf{x}_k, \mathbf{u}_k, t_k)} [g(\mathbf{x}_k, \mathbf{u}_k, \mathbf{x}_{k+1}, t_k) + V^*(\mathbf{x}_{k+1}, t_{k+1})]$$

Since the next state is random (probability given by $\mathbf{x}_{k+1} \sim f(\mathbf{x}_k, \mathbf{u}_k, t_k)$), we compute the *expected value*

$$\mathbb{E}_{x \sim p(x)}[f(x)] = \int_x p(x)f(x)dx = \sum_x p(x)f(x)$$

continuous discrete

The Bellman Equation and variants

Infinite horizon setting

What is a suitable problem setting where $N \rightarrow \infty$?

Time-invariant dynamics & cost, finite cost & discounted future cost

$$V^*(\mathbf{x}_k, t_k) = \min_{\mathbf{u}_k \in \mathcal{U}} g(\mathbf{x}_k, \mathbf{u}_k) + \gamma V^*(f(\mathbf{x}_k, \mathbf{u}_k), t_{k+1}) \quad 0 < \gamma \leq 1$$

Dynamic programming algorithm: Start with terminal cost:
 $V^*(\mathbf{x}_N, t_N) = g_N(\mathbf{x}_N)$. But $N \rightarrow \infty \dots$ 🤔

$$V^*(\mathbf{x}_k) = \min_{\mathbf{u}_k \in \mathcal{U}} g(\mathbf{x}_k, \mathbf{u}_k) + \gamma V^*(f(\mathbf{x}_k, \mathbf{u}_k))$$

The Bellman Equation and variants

Continuous time setting

$$J(\mathbf{x}(0), u(\cdot), T) = g_T(\mathbf{x}(T)) + \int_0^T g(\mathbf{x}(\tau), u(\tau), \tau) d\tau$$

Approximate as discrete-time with small timestep $\epsilon \rightarrow 0$.

Stage cost: $\epsilon g(\mathbf{x}(\tau), u(\tau), \tau)$

$$V^*(\mathbf{x}(t), t) = \min_{\mathbf{u}(t) \in \mathcal{U}} \epsilon g(\mathbf{x}(t), \mathbf{u}(t), t) + V^*(\mathbf{x}(t + \epsilon), t + \epsilon)$$

Continuous-time derivation cont'd

$$V^*(\mathbf{x}(t), t) = \min_{\mathbf{u}(t) \in \mathcal{U}} \epsilon g(\mathbf{x}(t), \mathbf{u}(t), t) + V^*(\mathbf{x}(t + \epsilon), t + \epsilon)$$

(First order Taylor expansion)

$$V^*(\mathbf{x}(t), t) = \min_{\mathbf{u}(t) \in \mathcal{U}} \epsilon g(\mathbf{x}(t), \mathbf{u}(t), t) + V^*(\mathbf{x}(t), t) + \nabla V^*(\mathbf{x}(t), t)^T f(\mathbf{x}(t), \mathbf{u}(t), t) \epsilon + \frac{\partial V^*}{\partial t} \epsilon + \mathcal{O}(\epsilon^2)$$

(Divide by ϵ , then $\epsilon \rightarrow 0$)

$$0 = \min_{\mathbf{u}(t) \in \mathcal{U}} g(\mathbf{x}(t), \mathbf{u}(t), t) + \nabla V^*(\mathbf{x}(t), t)^T f(\mathbf{x}(t), \mathbf{u}(t), t) + \frac{\partial V^*}{\partial t} + \mathcal{O}(\epsilon)$$

Hamilton-Jacobi-Bellman equation

$$\frac{\partial V^*}{\partial t} + \min_{\mathbf{u}(t) \in \mathcal{U}} g(\mathbf{x}, \mathbf{u}, t) + \nabla V^*(\mathbf{x}, t)^T f(\mathbf{x}, \mathbf{u}, t) = 0$$

What if the stage cost was zero?

$$V^*(\mathbf{x}_k, t_k) = g_N(\mathbf{x}_N) + \sum_{k=k}^{N-1} g(\mathbf{x}_k, \mathbf{u}_k, t_k)$$

But why is it hard to solve in closed-form for continuous state/action

$$V^*(\mathbf{x}_k, t_k) = \min_{\mathbf{u}_k \in \mathcal{U}} g(\mathbf{x}_k, \mathbf{u}_k, t_k) + V^*(f(\mathbf{x}_k, \mathbf{u}_k, t_k), t_{k+1})$$

Are there any problems where we can
derive a closed-form solution?